模型—视图—控制器

MODELS • VIEWS • CONTROLLERS | 原文：Trygve Reenskaug，1979 年 12 月 10 日

模型（MODELS）

模型表示知识。模型可以只是一个对象（那样没什么趣味），也可以是某种由对象组成的结构。这里提出的实现方式支持以类似语义网的形式表示知识（如果我正确理解了 Laura 的意思的话）。

一方面是模型及其组成部分，另一方面是模型所有者所感知的、由模型表示的世界；两者之间应当存在一一对应的关系。因此，模型中的节点应当代表问题中一个能够明确辨认的部分。

模型中的所有节点都应处于同一问题层次。把面向问题的节点（例如日历中的约会事项）与实现细节（例如段落）混在一起，会导致理解上的问题，也被认为是不良的设计方式。

视图（VIEWS）

视图是其模型的一种（视觉）表示。通常，视图会突出模型的某些属性，而隐藏其他属性。因此，它起着呈现过滤器的作用。

视图与它的模型（或模型的一部分）相连接，通过向模型提问来获取呈现所需的数据。它也可以通过发送适当的消息来更新模型。所有这些提问和消息都必须使用模型自身的术语，因此，视图必须理解自己所表示的那些模型属性的语义。（例如，视图可以问模型的标识符，并期望得到一个 Text 类的实例；但它不能假定模型本身属于 Text 类。）

控制器（CONTROLLERS）

控制器是用户与系统之间的纽带。它让相关视图出现在屏幕上的合适位置，向用户提供信息。它也通过向用户提供选项列表或其他方式，使用户能够发出命令、提交数据。控制器接收用户发出的这些信息，将其转换为适当的消息，再把这些消息交给一个或多个视图。

控制器绝不应当补充视图的表现工作。例如，它绝不应当通过在节点的视图之间绘制箭头，来连接这些视图。

反过来，视图也绝不应当知道用户输入的事情，例如指针设备（mouse）操作和按键。我们应当始终能够在控制器中编写一个方法，让它向视图发送消息，从而精确重现用户的任意一组连续命令。

内容修改工具（EDITORS）

一个控制器与它的所有视图相连接；这些视图被称为该控制器的组成部分。有些视图会提供一种特殊的控制器——修改工具（editor），它允许用户修改视图所呈现的信息。这样的修改工具可以插入控制器和视图之间的路径中，充当控制器的扩展。修改过程一旦完成，修改工具就会从这条路径中移除并被丢弃。

请注意，修改工具是通过与之连接的视图所采用的交互表达方式来与用户交流的，因此，修改工具与该视图密切相关。控制器若想取得一个修改工具，应当向视图请求；没有其他合适的来源。

译者说明（不属于原文）

原文中的“向用户提供输入（input）”和“用户输出（output）”是从用户接收信息、用户向系统发出信息的角度表述的；译文用“向用户提供信息”和“用户发出命令、提交数据”表达同一含义。

这份 1979 年文献明确写到：控制器将消息交给视图，视图可以更新模型；文中并未提出后来常见的“模型主动通知视图”的观察者机制。因此不应把后续 MVC 版本的具体机制直接代入这份原始说明。